

Lecture 9, 10, 11, 12 -- Machine Learning

机器学习

- 定义：若一个计算机程序在某类任务 T 上，其关于性能度量 P 的表现会随着经验 E 的增加而改善，则称该程序从经验 E 中学习 (learn)。

Given: Training data: $(x_1, y_1), \dots, (x_n, y_n)$, $x_i \in \mathbb{R}^d$ and y_i is the label.

example $x_1 \rightarrow$	x_{11}	x_{12}	$\dots$	x_{1d}	$y_1 \leftarrow \text{label}$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
example $x_i \rightarrow$	x_{i1}	x_{i2}	$\dots$	x_{id}	$y_i \leftarrow \text{label}$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
example $x_n \rightarrow$	x_{n1}	x_{n2}	$\dots$	x_{nd}	$y_n \leftarrow \text{label}$

fruit	length	width	weight	label
fruit 1	165	38	172	Banana
fruit 2	218	39	230	Banana
fruit 3	76	80	145	Orange
fruit 4	145	35	150	Banana
fruit 5	90	88	160	Orange
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
fruit n	$\dots$	$\dots$	$\dots$	$\dots$

11

- 无监督学习：**从无标签 (unlabeled) 的数据中学习模型。

Training data: "examples" x .

$$x_1, \dots, x_n, x_i \in X \subset \mathbb{R}^d$$

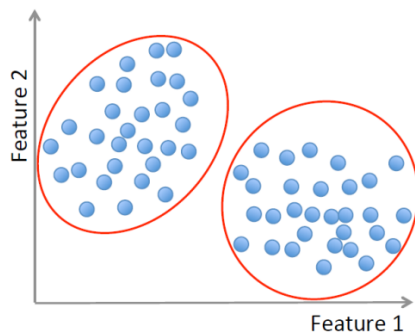
聚类/分割

- Clustering/segmentation:**

目标函数 $f: \mathbb{R}^d \rightarrow \{C_1, \dots, C_k\}$ (set of clusters).

Example: Find clusters in the population, fruits, species.

- 二维特征空间中的两簇示意



- 方法:

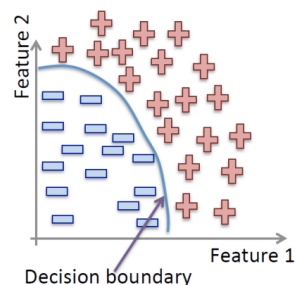
- K-means (K 均值)
- 高斯混合 (Gaussian mixtures)
- 层次聚类 (hierarchical clustering)
- 谱聚类 (spectral clustering)

- 有监督学习：**从有标签 (labeled) 的数据中学习模型。

Training data: "examples" x with "labels" y .
 $(x_1, y_1), \dots, (x_n, y_n), x_i \in \mathbb{R}^d$

- **Classification:** y is discrete. To simplify, $y \in \{-1, +1\}$ 二元分类器
 $f: \mathbb{R}^d \rightarrow \{-1, +1\}$ (f is called a **binary classifier**)
 Example: Approve credit yes/no, spam/ham, banana/orange.

- 二维特征下的样本分布与决策边界（把样本分成两类）。



- 方法 (Methods) :

- **支持向量机 (Support Vector Machines)**

- 主要基于三点思想：
 - 大间隔分类
 - 正则化（用于数据不可线性分隔的情况）
 - 特征变换与核函数（用于超越线性分类器）
- 线性 SVM

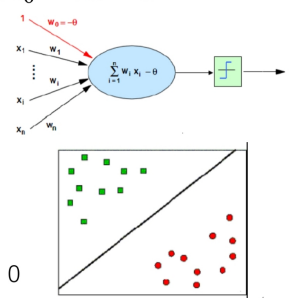
权重向量 $\mathbf{w}$, 样本的特征向量 $\mathbf{x}$, 偏置, 用于平移分类超平面, 使其不必过原点
 $\mathbf{w} = (w_1, \dots, w_d), \mathbf{x} = (x_1, \dots, x_d)$ and $b = -\theta$

分类器的输出 (± 1)
 $y = \text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle + b)$
 $\langle \mathbf{w}, \mathbf{x} \rangle$ 与 $\mathbf{x}$ 的内积 (点积)

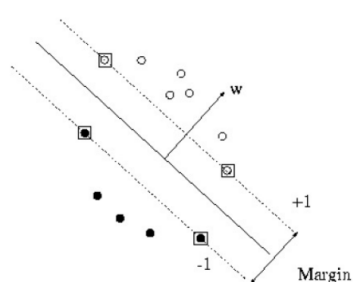
- the decision boundary is the hyperplane

$$f(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b = 0$$

- decision rule: assign $\mathbf{x}$ to class 1 iff $f(\mathbf{x}) \geq 0$



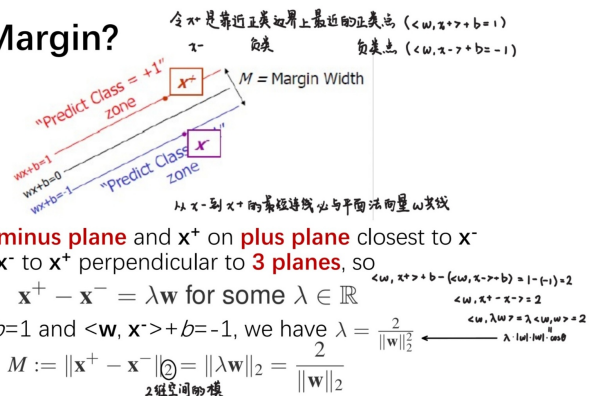
- 最优间隔分类器:



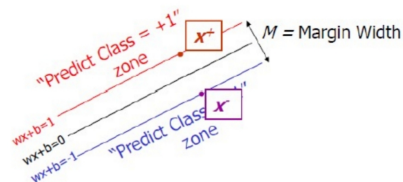
- 寻找具有最大间隔的分类器

- 最大化数据点到决策边界的最小距离
- 称为线性支持向量机 linear support vector machines
- 支持向量 support vectors: 与间隔边界接触的数据点
- 数学形式说明:
 - 决策边界 decision boundary : $\mathbf{w} \cdot \mathbf{x} + b = 0$
 - plus-plane (正平面) : 与决策边界平行、接触一些正样本的超平面, 形式为 $\mathbf{w} \cdot \mathbf{x} + b = c$ (某常数 c)
 - minus-plane (负平面) : 与决策边界平行、接触一些负样本的超平面, 形式为 $\mathbf{w} \cdot \mathbf{x} + b = -c$, 且决策边界位于 plus 与 minus 平面中间 (半路位置)
 - 对上述两个平面同时除以常数 c (平面位置不变), 并把 w/c 重命名为 w , b/c 重命名为 b , 则得到规范化形式:
 - 决策边界: $\mathbf{w} \cdot \mathbf{x} + b = 0$
 - plus-plane: $\mathbf{w} \cdot \mathbf{x} + b = 1$
 - minus-plane: $\mathbf{w} \cdot \mathbf{x} + b = -1$
 - w 与这三条平面垂直:
 - 对决策边界上的任意两点 u 和 v , 都有 $\mathbf{w} \cdot u + b = 0$ 与 $\mathbf{w} \cdot v + b = 0$, 两式相减得 $\mathbf{w} \cdot (u - v) = 0$, 说明任意两个决策边界上的点之差与 w 正交, 因此 w 为平面的法向量 (垂直于平面)。

What is Margin?



Optimal Margin Classifier



- training set $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$
- find w and b to

$$\max \frac{2}{\|\mathbf{w}\|_2} \quad \text{s.t.} \quad \langle \mathbf{w}, \mathbf{x}_i \rangle + b \begin{cases} \geq 1, & \text{if } y_i = 1 \\ \leq -1, & \text{if } y_i = -1. \end{cases} \quad (i = 1, \dots, n)$$

equivalent constrained optimization problem: find $\mathbf{w}, b$ to

$$\min_{\mathbf{w}} \frac{\|\mathbf{w}\|_2^2}{2} \quad \text{subject to } y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1, \forall i$$

- 每个数据点对应一个约束
- 一个二次规划 (QP) 问题：有商业软件可以求解
- 但是，我们将研究对偶优化问题
 - 使支持向量机在高维数据上高效工作
 - 在处理非线性可分的数据集时这是必要的
- 拉格朗日 Lagrangian
 - 原始问题的拉格朗日函数为：

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{\|\mathbf{w}\|_2^2}{2} - \sum_{i=1}^n \alpha_i (y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1)$$

- 其中 $\alpha = (\alpha_1, \dots, \alpha_n) \geq 0$ 是拉格朗日乘子 Lagrangian multipliers
- 强对偶性 strong duality :
 - 若数据线性可分 linearly separable , 例如存在满足所有约束的 $\mathbf{w}$ 和 b , 则有：

$$\max_{\alpha: \alpha_i \geq 0} \min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \alpha) = \min_{\mathbf{w}, b} \max_{\alpha: \alpha_i \geq 0} \mathcal{L}(\mathbf{w}, b, \alpha)$$

- 对偶优化问题：
 - 对于给定的 α , 定义

$$\mathcal{L}_d(\alpha) = \min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \alpha)$$

- 对偶优化问题为：

$$\max_{\alpha: \alpha_i \geq 0} \mathcal{L}_d(\alpha) = \max_{\alpha: \alpha_i \geq 0} \min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \alpha)$$

- 对于固定的 α , 先求解 $\min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \alpha)$: 求偏导得必要条件：

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \\ \frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0 \end{cases} \implies \begin{cases} \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \\ \sum_{i=1}^n \alpha_i y_i = 0 \end{cases}$$

- 将最优 $\mathbf{w}$ 代入并固定 α 后的拉格朗日，得到以对偶变量 α 表示的对偶问题：

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

$$\text{s.t. } \alpha_i \geq 0, \sum_{i=1}^n \alpha_i y_i = 0$$

- 可以用任意通用的数值优化软件包求解，例如 MATLAB 优化工具箱
- 由于问题是凸的，可找到全局最优解

- Support Vectors:

- KKT 互补性条件:

$$\alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1] = 0$$

- 对于满足 $y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) > 1$ 的样本， $\alpha_i = 0$ （约束不活跃）： $\mathbf{x}_i$ 对解无关紧要。
- 对于有 $\alpha_i > 0$ 的样本（约束活跃），有 $y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = 1$ ：这些样本位于间隔边界上。
- 解由位于边界上的样本决定（support vectors）：如果移除所有其它训练点并重新训练，仍会得到相同的超平面。

- Find b:

- 求解二次规划（QP）问题可得到最优的 α^* 与 $\mathbf{w}$:

$$\mathbf{w}(\alpha^*) = \sum_{i \in S} \alpha_i^* y_i \mathbf{x}_i$$

- 其中 S 是支持向量的集合

$$S = \{i : \alpha_i^* > 0, i = 1, \dots, n\}$$

- 再次利用 KKT 互补性条件:

- 任一支持向量 $(\mathbf{x}_s, y_s)$ 满足

$$y_s (\langle \mathbf{w}(\alpha^*), \mathbf{x}_s \rangle + b(\alpha^*)) = 1$$

- 由此可得到 $b(\alpha^*)$ 的表达式:

$$b(\alpha^*) = \frac{1}{|S|} \sum_{s \in S} \left(\frac{1}{y_s} - \sum_{i \in S} \alpha_i^* y_i \langle \mathbf{x}_i, \mathbf{x}_s \rangle \right)$$

- 预测“新样本 $\mathbf{x}$ 属于哪个类别”的规则为:

$$\text{sign}(\langle \mathbf{w}(\alpha^*), \mathbf{x} \rangle + b(\alpha^*))$$

- recall that $\mathbf{w}(\alpha) = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$, then

$$\text{sign}(\langle \mathbf{w}(\alpha^*), \mathbf{x} \rangle + b(\alpha^*)) = \text{sign}\left(\sum_{i \in S} \alpha_i^* y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + b(\alpha^*)\right)$$

- 数据不可线性分隔 / 正则化

When Data Not Linearly Separable

- if data **linearly separable**, find a plane that separates the two class with 0 error

$$\min_{\mathbf{w}} \frac{\|\mathbf{w}\|_2^2}{2} \quad \text{s.t. } y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1, \forall i$$

- if data **not linearly separable**, try to find a plane separating two classes with **minimal** errors **松弛变量**
- introduce positive **slack variables** ξ_i , the summation of which is an upper bound on the number of **training errors**

$$y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i \quad \xi_i \geq 0 \quad \forall i$$

- penalize** $\sum_i \xi_i$ in the objective function

$$\min_{\mathbf{w}, \xi_i \geq 0} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t. } y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i$$

- the larger the constant C , the more we want to minimize error, the more complex the **decision boundary**

19

Lagrangian

- Lagrangian**: with dual variables $\alpha_i \geq 0, \mu_i \geq 0$

$$\mathcal{L}(\mathbf{w}, b, \xi, \alpha, r) = \frac{\|\mathbf{w}\|_2^2}{2} + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i (y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) - 1 + \xi_i) - \sum_{i=1}^n \mu_i \xi_i$$

- Solving the dual: $\min_{\mathbf{w}, b, \xi} \mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mu)$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \implies \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{i=1}^n \alpha_i y_i = 0$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \implies C - \alpha_i - \mu_i = 0$$

Dual Problem

- Dual**: still a **QP** problem: $\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$

$$\text{s.t. } \sum_{i=1}^n \alpha_i y_i = 0 \text{ and } 0 \leq \alpha_i \leq C, \forall i$$

- KKT condition**

$$\begin{cases} \alpha_i \geq 0, \mu_i \geq 0 \\ \xi_i \geq 0, \mu_i \xi_i = 0, \end{cases}$$

$$y_i f(\mathbf{x}_i) - 1 + \xi_i \geq 0,$$

$$\alpha_i (y_i f(\mathbf{x}_i) - 1 + \xi_i) = 0.$$

► $\forall (\mathbf{x}_i, y_i)$, either $\alpha_i = 0$ or $y_i f(\mathbf{x}_i) = 1 - \xi_i$

► $\alpha_i = 0 \Rightarrow (\mathbf{x}_i, y_i)$ has no influence on f

► $\alpha_i > 0 \Rightarrow y_i f(\mathbf{x}_i) = 1 - \xi_i$ **support vector**

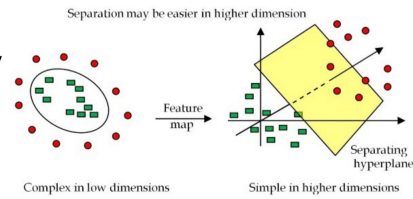
► $\alpha_i < C \Rightarrow \mu_i > 0$ and $\xi_i = 0$ (lie on margin)

► $\alpha_i = C \Rightarrow \mu_i = 0$. moreover, if $\xi_i \leq 1$, $(\mathbf{x}_i, y_i)$ lie within margin, otherwise misclassified

the prediction model only depend on support vectors!

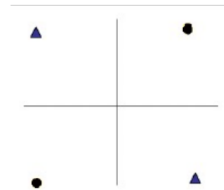
- 非线性 SVM / 核函数

Nonlinear Decision Boundary & Feature Transformation



- mapping from the input space $\mathbb{R}^d$ (**attributes**) to a feature space $\mathcal{H}$ (**features**) $\psi: \mathbb{R}^d \rightarrow \mathcal{H}, \mathbf{x} \rightarrow \psi(\mathbf{x})$ 将输入空间映射到特征空间
- transform the data with the mapping
 $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n) \rightarrow (\psi(\mathbf{x}_1), y_1), \dots, (\psi(\mathbf{x}_n), y_n)$
- we have **linear decision boundary** on feature space 线性决策边界
- 通常，特征空间维度越高，数据越可能变得线性可分（更容易线性分离）

Example



- data
 $(x_1, y_1; y) : (-1, -1; -1), (-1, 1; +1), (1, -1; +1), (1, 1; -1)$
- the data set is **not** linearly separable
- however, if we transform the data using
 $(x_1, x_2; y) \rightarrow (x_1, x_2, (x_1 x_2); y)$
 $(-1, -1, 1; -1), (-1, 1, -1; +1), (1, -1, -1; +1), (1, 1, 1; -1)$
- **linearly separable**: $x_1 x_2 > 0 \Rightarrow -1, x_1 x_2 \leq 0 \Rightarrow +1$

Apply SVM After Feature Transformation

- **Dual Problem on Features:**

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle \psi(\mathbf{x}_i), \psi(\mathbf{x}_j) \rangle \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \text{ and } 0 \leq \alpha_i \leq C, \forall i \end{aligned}$$

$\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ replaced by $\langle \psi(\mathbf{x}_i), \psi(\mathbf{x}_j) \rangle$!

核技巧 (Kernel Trick)

- Define $k(\mathbf{x}_i, \mathbf{x}_j) = \langle \psi(\mathbf{x}_i), \psi(\mathbf{x}_j) \rangle$ called **kernel function**

- Rewrite the problem as

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \text{ and } 0 \leq \alpha_i \leq C, \forall i \end{aligned}$$

- **kernel trick**

- **no need to explicitly calculate ψ**
- **dot product** $\langle \psi(\mathbf{x}_i), \psi(\mathbf{x}_j) \rangle$ realized by the kernel function $k(\mathbf{x}_i, \mathbf{x}_j)$
- k is cheaper to calculate
- allow one to use very high dimensional feature space

Common Kernels

- **linear kernel**: $k(\mathbf{x}, \tilde{\mathbf{x}}) = \langle \mathbf{x}, \tilde{\mathbf{x}} \rangle$, **identity mapping**
- **polynomial kernel**: $k(\mathbf{x}, \tilde{\mathbf{x}}) = \langle \mathbf{x}, \tilde{\mathbf{x}} \rangle^m$, corresponding to feature transformation
$$\psi(\mathbf{x}) = (x_1x_1, x_1x_2, \dots, x_1x_n, \dots, x_nx_1, x_nx_2, \dots, x_nx_n)$$
- **inhomogeneous polynomial**: $k(\mathbf{x}, \tilde{\mathbf{x}}) = (\langle \mathbf{x}, \tilde{\mathbf{x}} \rangle + 1)^m$
- **Gaussian kernel**: $k(\mathbf{x}, \tilde{\mathbf{x}}) = \exp(-\|\mathbf{x} - \tilde{\mathbf{x}}\|_2^2 / (2\sigma^2))$
 - **radial basis function** (RBF) network
 - corresponding to an **infinite-dimensional feature space**

Any algorithm that depends only on dot products can use the kernel trick!

27

- 核函数的直觉与 Mercer 条件 (Mercer condition)
 - 直觉上, $k(x, \tilde{x})$ 表示我们对样本 x 与 $\tilde{x}$ 相似性的先验知识。
 - 要存在某个映射 ψ , 使 $k(x, \tilde{x}) = \langle \psi(x), \psi(\tilde{x}) \rangle$, k 需要满足一个技术条件 (即 Mercer 条件)。
 - Mercer 条件: 存在一个希尔伯特空间 $\mathcal{H}$, 使得 k 在该空间中定义了一个点积。
 - 等价判据: 对于任意样本集 $D = \{x_1, \dots, x_n\}$, 对应的核矩阵 K ($K_{ij} = k(x_i, x_j)$) 必须是正半定 (positive semidefinite)。

- **Mercer condition** for k to be a kernel function
 - there is a Hilbert space $\mathcal{F}$ for which k defines a **dot product**
 - the above is true if k is a **positive semidefinite function**: K is positive semi-definite for any $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$

$$\mathbf{K} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_j) & \cdots & k(\mathbf{x}_1, \mathbf{x}_n) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_i, \mathbf{x}_1) & \cdots & k(\mathbf{x}_i, \mathbf{x}_j) & \cdots & k(\mathbf{x}_i, \mathbf{x}_n) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & \cdots & k(\mathbf{x}_n, \mathbf{x}_j) & \cdots & k(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix}$$

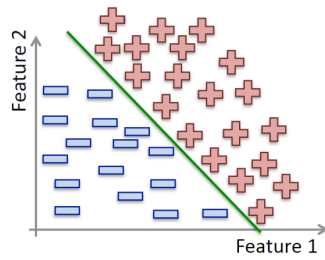
- 用 SVM 进行分类
 - 选择一个非线性变换 $\psi: \mathbb{R}^d \rightarrow \mathcal{H}$, $x \mapsto \psi(x)$
 - (通过核函数隐式实现: $k(x, \tilde{x}) = \langle \psi(x), \psi(\tilde{x}) \rangle$)
 - 在特征空间上求解对偶优化问题, 得到 α^* 。
 - 对未来样本进行分类如下:

$$\text{sign}\left(\sum_{i=1}^n \alpha_i^* y_i k(\mathbf{x}_i, \mathbf{x}) + b(\alpha^*)\right)$$

- 只需计算核函数 k 即可满足训练与预测所需 (不必显式计算 ψ)。

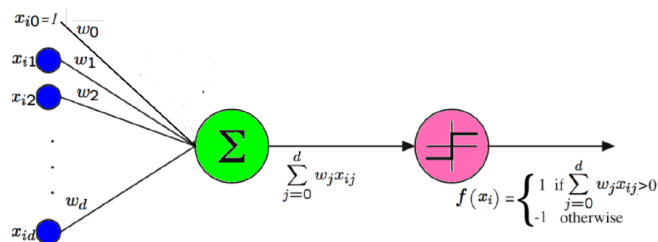
- **神经网络 (neural networks)**

- **感知器 perceptron**: 最简单的分类方法, 最简单的神经网络。



- 如果数据线性可分，感知器可完美工作；否则不会收敛。
- 给定 n 个样本和 d 个特征：

$$f(x_i) = \text{sign}\left(\sum_{j=0}^d w_j x_{ij}\right)$$



- 思路：从一个随机超平面开始，利用训练数据不断调整它。迭代方法。
-

Perceptron Algorithm

Input: A set of examples, $(x_1, y_1), \dots, (x_n, y_n)$

Output: A perceptron defined by $(w_0, w_1, \dots, w_d)$

Begin

2. Initialize the weights w_j to 0 $\forall j \in \{0, \dots, d\}$
3. Repeat until convergence
4. For each example $x_i \forall i \in \{1, \dots, n\}$
5. if $y_i f(x_i) \leq 0$ #an error?
6. update all w_j with $w_j := w_j + y_i x_{ij}$ #adjust the weights

End

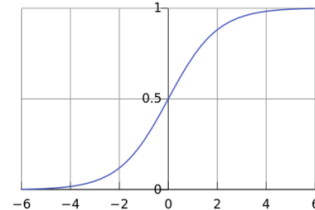
- 权重 $w_1, \dots, w_d$ 决定决策边界的斜率。 w_0 决定决策边界的偏移（有时记作 b ）。
- 算法第 6 行的含义：
 - 对正例判断错误时：把 x 加到权重向量上。
 - 对负例判断错误时：从权重向量中减去 x 。
 - 其他变体会加或减 ηx (η 为学习率)。
- 当权重不再变化（最后两个权重向量之差为 0）时，算法收敛。
- 权重 w_j 决定特征 x_j 对标签的贡献。

- $-w_0$ 是一个阈值量，要求 $\sum_{j=1}^d w_j x_j$ 超过该值时感知器才输出 1。
- 神经网络利用感知器表示基本函数的能力，并将这些基本函数按层组合成网络。
- 但感知器使用阶跃函数（step function），阶跃函数不可导；当数据不可线性分离时，不适合用梯度下降法。
- 我们需要一个对输入可微的输出函数，其中一种选择是使用 Sigmoid 函数：

$$g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}$$

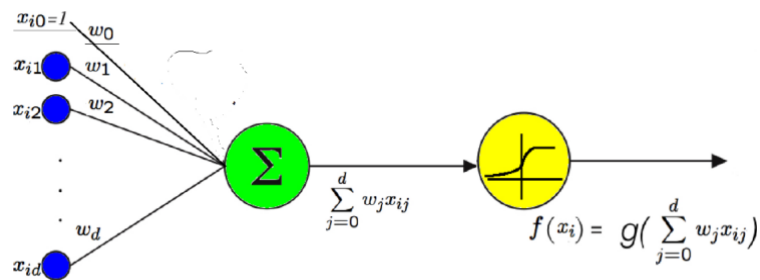
$$g(z) \rightarrow 1 \text{ when } z \rightarrow +\infty$$

$$g(z) \rightarrow 0 \text{ when } z \rightarrow -\infty$$



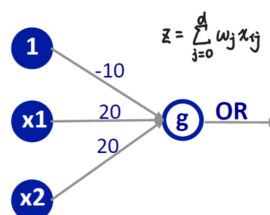
- 给定 n 个样本和 d 个特征，对于样本 x_i （样本矩阵的第 i 行）：

$$f(x_i) = \frac{1}{1 + e^{-\sum_{j=0}^d w_j x_{ij}}}$$

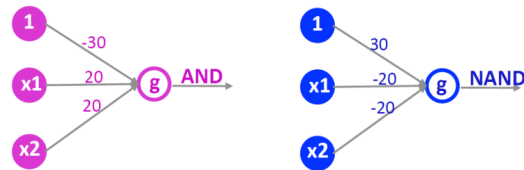


The XOR example

x_1	x_2	x_1 OR x_2	$g(z)$
0	0	0	$g(w_0 + w_1 x_1 + w_2 x_2) = g(-10)$
0	1	1	$g(10)$
1	0	1	$g(10)$
1	1	1	$g(30)$



Similarly, we obtain the perceptrons for the AND and NAND:

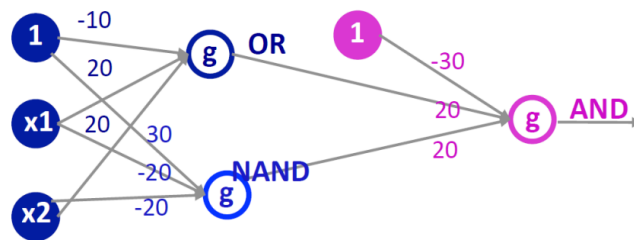


Note: how the weights in the NAND are the inverse weights of the AND.

Let's try to create a NN for the XOR function using elementary perceptrons.

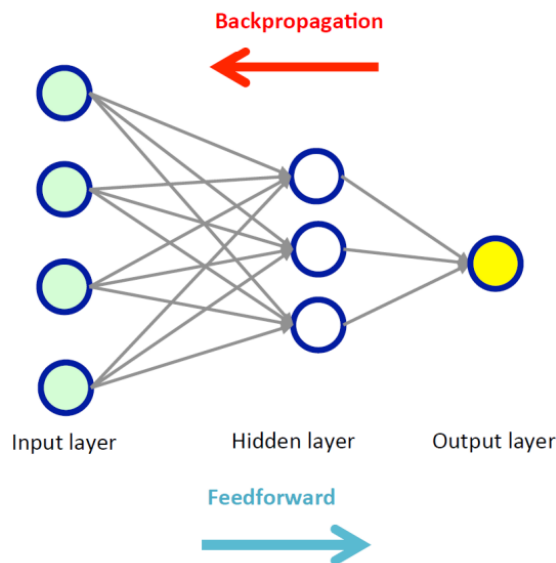
x_1	x_2	$x_1 \text{ XOR } x_2$	$(x_1 \text{ OR } x_2) \text{ AND } (x_1 \text{ NAND } x_2)$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

Let's put them together...



XOR as a combination of 3 basic perceptrons.

- Feedforward-Backpropagation



- 反向传播算法 (BP)

- 前馈神经网络 Feedforward NN (与循环网络 recurrent networks 相对) 没有回路连接。
 - 学习多层网络的权重。
 - 反向传播 (backpropagation) 指“误差的向后传播”。

- 给定一个结构固定的网络（神经元与互连关系固定）。
- 使用梯度下降最小化网络输出 o 与真实标签 y 之间的平方误差。
- 假设有多个输出 k 。
- 挑战：在网络中搜索所有神经元的所有可能权重值。
- 输入：训练样本 (x, y) 、学习率 α （例如 $\alpha = 0.1$ ）、输入层、隐藏层和输出层的神经元数 n_i 、 n_h 和 n_o 。
- 输出：一个神经网络，包含一层输入层、一层隐藏层和一层输出层，分别具有 n_i 、 n_h 和 n_o 个神经元，并包含其所有权重。
- 创建前馈网络 (n_i, n_h, n_o)
- 将所有权重初始化为小的随机数（例如在 $[-0.2, 0.2]$ ）
- 重复直到收敛
 - 对每个训练样本 (x, y) :
 - I. 正向传播：将样本 x 通过网络传播，并计算每个神经元的输出 o_j
 - II. 反向传播：将误差向后传播。
 - 情形 1：对每个输出神经元 k ，计算其误差

$$\delta_k = o_k(1 - o_k)(y_k - o_k)$$
 - 情形 2：对每个隐藏神经元 h ，计算其误差

$$\delta_h = o_h(1 - o_h) \sum_{k \in \text{Succ}(h)} w_{hk} \delta_k$$
 - III. 更新每个权重：

$$w_{ij} \leftarrow w_{ij} + \alpha \delta_j x_{ij}$$

- Backpropagation rules:

- 对于由 (x, y) 定义的训练样本 e ，该样本在网络上所有输出神经元上的误差之和为：

$$E_e(w) = \frac{1}{2} \sum_k (y_k - o_k)^2$$

- 记住，梯度下降对所有训练样本逐个迭代，沿着该样本误差关于权重的梯度方向下降。权重更新规则（梯度下降）：

$$\Delta w_{ij} = -\alpha \frac{\partial E_e(w)}{\partial w_{ij}}$$

- 使用链式法则：

$$\frac{\partial E_e}{\partial w_{ij}} = \frac{\partial E_e}{\partial z_j} \frac{\partial z_j}{\partial w_{ij}} = \frac{\partial E_e}{\partial z_j} x_{ij}$$

- 因此权重更新可写为：

$$\Delta w_{ij} = -\alpha \frac{\partial E_e}{\partial z_j} x_{ij}$$

- 在计算 $\partial E_e / \partial z_j$ 时，考虑两种情况：

- 情况 1：神经元 j 是输出层神经元。

Backpropagation rules

- Case 1: Neuron j is an output neuron $\frac{\partial E}{\partial z_j} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial z_j}$

$$\frac{\partial E}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_k (y_k - o_k)^2$$

$$\frac{\partial E}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} (y_j - o_j)^2$$

$$\frac{\partial E}{\partial o_j} = \frac{1}{2} \cdot 2 (y_j - o_j) \frac{\partial (y_j - o_j)}{\partial o_j}$$

$$\frac{\partial E}{\partial o_j} = -(y_j - o_j)$$

We have: $o_j = g(z_j)$

$$\frac{\partial o_j}{\partial z_j} = \frac{\partial g(z_j)}{\partial z_j} = \frac{\partial \frac{1}{1+e^{-z_j}}}{\partial z_j} = -\frac{-e^{-z_j}}{(1+e^{-z_j})^2}$$

$$= \frac{1}{1+e^{z_j}} \cdot \left(1 - \frac{1}{1+e^{z_j}}\right) = g(z_j) \cdot (1 - g(z_j)) = o_j(1 - o_j)$$

$$\frac{\partial o_j}{\partial z_j} = o_j(1 - o_j)$$

40

$$\frac{\partial E}{\partial z_j} = -(y_j - o_j) o_j(1 - o_j)$$

$$\Delta w_{ij} = \alpha (y_j - o_j) o_j(1 - o_j) x_{ij}$$

We will note

$$\delta_j = -\frac{\partial E}{\partial z_j}$$

$$\Delta w_{ij} = \alpha \delta_j x_{ij}$$

- 情况 2：神经元 j 是隐藏层神经元。

- Case 2: Neuron j is a hidden neuron

$$\frac{\partial E}{\partial z_j} = \sum_{k \in \text{succ}\{j\}} \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial z_j} = \sum_{k \in \text{succ}\{j\}} -\delta_k \frac{\partial z_k}{\partial z_j}$$

$$\frac{\partial E}{\partial z_j} = \sum_{k \in \text{succ}\{j\}} -\delta_k \left[\frac{\partial z_k}{\partial o_j} \frac{\partial o_j}{\partial z_j} \right]$$

$$\frac{\partial E}{\partial z_j} = \sum_{k \in \text{succ}\{j\}} -\delta_k w_{jk} \frac{\partial o_j}{\partial z_j}$$

$$\frac{\partial E}{\partial z_j} = \sum_{k \in \text{succ}\{j\}} -\delta_k w_{jk} o_j (1 - o_j)$$

$$\delta_j = -\frac{\partial E}{\partial z_j} = o_j (1 - o_j) \sum_{k \in \text{succ}\{j\}} \delta_k w_{jk}$$

42

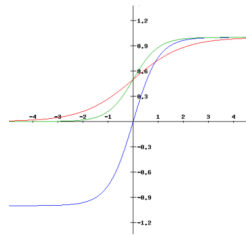
- 反向传播 符号说明：

- x_{ij} ：第 i 个输入到神经元 j 的值。

- w_{ij} : 与神经元 j 的第 i 个输入对应的权重。
- $z_j = \sum w_{ij}x_{ij}$: 神经元 j 的加权输入和。
- o_j : 神经元 j 的输出。
- g : sigmoid 激活函数。
- outputs: 输出层中神经元的集合。
- Succ(j): 那些其输入直接包含神经元 j 输出的神经元的集合 (即 j 的后继)。

• 观察:

- 收敛: 权重的变化很小
- 还有其他激活函数。双曲正切函数 (tanh) 在实际中对神经网络更好, 因为其输出范围为 -1 到 1 。

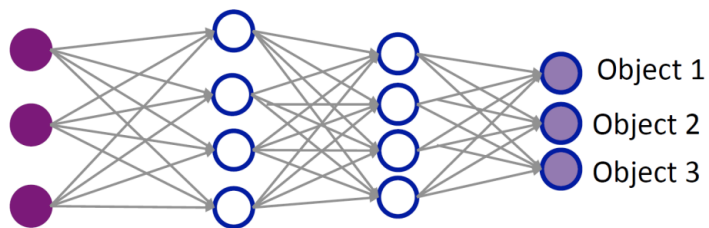


$$g(x) = \text{sigmoid}(x) = \frac{e^{kx}}{1 + e^{kx}} \quad \text{for } k = 1, k = 2, \text{ etc.}$$

$$g(x) = \text{tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (\text{It is a rescaling of the logistic/sigmoid function!})$$

• 多类情况 multi-class case:

- 现在, 具有超过两层的网络 (亦称深度网络) 已在许多领域被证明非常有效。



- 深度网络的例子: 受限玻尔兹曼机、卷积神经网络、自编码器等。

• 决策树 (decision trees)

• K 最近邻 (K-nearest neighbors) KNN

- 主要思想: 利用样本之间的相似性。
- 假设: 两个相似的样本应该有相同的标签。
- 假定所有样本 (实例) 都是位于 d 维空间 R^d 的点。
- KNN 使用标准的欧氏距离 (Euclidean distance) 来定义最近邻。给定两个样本 x_i 和 x_j :

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

- 训练算法 Training algorithm :

- 将每个训练样本 (x, y) 加入数据集 D 。 $x \in \mathbb{R}^d$, $y \in \{-1, +1\}$ 。

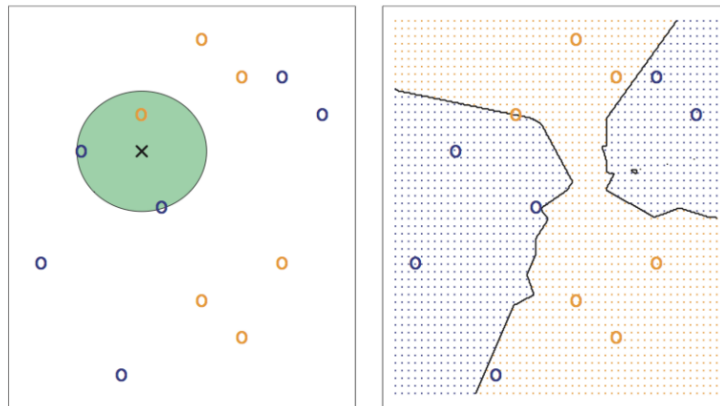
- 分类算法 Classification algorithm:

- 给定一个待分类样本 x_q 。设 $N_k(x_q)$ 为 x_q 的 k 个最近邻的集合。
- 预测公式:

$$\hat{y}_q = \text{sign}\left(\sum_{x_i \in N_k(x_q)} y_i\right)$$

- 说明: 对 k 个最近邻的标签求和, 然后取符号 (正则预测为 +1, 负则为 -1) 。
- 补充: 当求和为 0 (即正负票数相等) 时需要一个平局处理策略, 例如选择奇数 k 避免平局, 或按距离加权, 或随机选择/使用先验

-



- 优点 (Pros) :

- 易实现。在实践中表现良好 (有效) 。
- 不需要显式构建复杂模型、做很多假设或大量调参。
- 可以很容易地通过加入新样本来扩展 (增量加入训练样本即生效) 。

- 缺点 (Cons) :

- 需要大量存储空间来保存整个训练数据集 (因为预测时直接使用训练集) 。
- 慢! 给定 n 个样本和 d 个特征, 方法运行一次预测的时间复杂度为 $O(n \times d)$ (需要计算到所有训练点的距离并排序/取前 k) 。

- 易受“维度灾难”影响（当特征维度很高时，距离的判别能力下降，方法效果变差）。
- K-NN 的应用：信息检索（Information retrieval）。在大型数据库中使用最近邻进行手写字符分类。推荐系统（例如“与你相似的用户可能喜欢相似的电影”）。乳腺癌诊断。医学数据挖掘（例如：寻找具有相似症状的病人）。一般的模式识别。
- **朴素贝叶斯 (naive Bayes)**
- **逻辑回归 Logistic Regression :**
 - 给定：训练数据： $(x_1, y_1), \dots, (x_n, y_n)$, $x_i \in \mathbb{R}^d$, 且 y_i 为离散类别（分类/定性）， $y_i \in Y$ 。
 - 示例： $Y = \{-1, +1\}$, 或 $Y = \{0, 1\}$ 。
 - 任务：学习一个分类函数 $f: \mathbb{R}^d \rightarrow Y$ 。
 - 线性分类 Linear Classification: 若分类模型可由线性函数（线性超平面 linear hyperplane）表示，则称为线性分类。
 - 线性函数 $\beta_0 + \beta_1 x$ 的输出是任意实数，不能直接当作概率。sigmoid (S 型) 函数把这个线性分数压缩到 0 到 1 之间，因此常用来把线性模型的输出解释为“属于某一类的概率”。

Classification

$$y = f(x) = \beta_0 + \beta_1 x$$

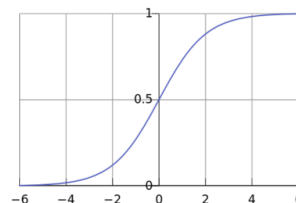
$$\text{Default} = \beta_0 + \beta_1 \times \text{Balance}$$

We want $0 \leq f(x) \leq 1$; $f(x) = P(Y = 1|x)$

We use the sigmoid function: $g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}$

$g(z) \rightarrow 1$ when $z \rightarrow +\infty$

$g(z) \rightarrow 0$ when $z \rightarrow -\infty$



- z (常写为 $z = \beta_0 + \beta_1 x$) 叫做 logit 或线性得分， $\text{sigmoid}(g(z))$ 就是该得分对应的概率。

Logistic Regression

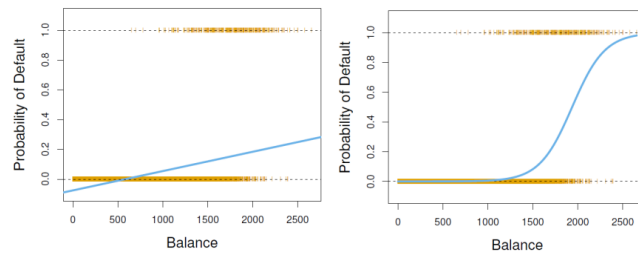
$$g(\beta_0 + \beta_1 x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$

$$\text{New } f(x) = g(\beta_0 + \beta_1 x)$$

In general:

$$f(x) = g\left(\sum_{j=1}^d \beta_j x_j\right)$$

- 这就是“逻辑回归”的核心：线性部分决定分界方向（超平面），sigmoid 部分把分数变成概率并引入非线性映射以保证输出在 $[0, 1]$ 。



Logistic regression is not a regression method but a classification method!

How to make a prediction?

- Suppose $\beta_0 = -10.65$ and $\beta_1 = 0.0055$. What is the probability of default for a customer with \$1,000 balance?

$$P(\text{default} = \text{yes} | \text{balance} = 1000) = \frac{1}{1 + e^{10.65 - 0.0055 \cdot 1000}}$$

$$P(\text{default} = \text{yes} | \text{balance} = 1000) = 0.00576$$

- To predict the class:

If $g(z) \geq 0.5$ predict $y = 1$ ($z \geq 0$)

If $g(z) < 0.5$ predict $y = 0$ ($z < 0$)

How to find the β 's?

$$R(\beta) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (f(x) - y)^2$$

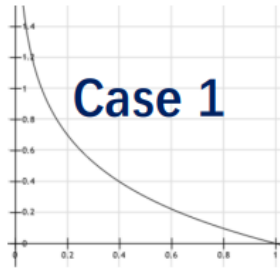
$$Loss = \frac{1}{2} (f(x) - y)^2$$

- 现在 $f(x)$ 是 logistic 函数 (sigmoid) , 因此 $(f(x) - y)^2$ 不再是我们在线性 f 情况下得到的二次函数。
- 损失/风险变成了复杂的非线性函数。
- 可能有许多局部极小点, 因此直接用梯度下降不一定能找到全局最优。
- 我们需要一个新的凸函数来替代 (便于优化、保证全局最优) 。

New Convex function: $Cost(f(x), y) = \begin{cases} -\log(f(x)) & \text{if } y = 1 \\ -\log(1 - f(x)) & \text{if } y = 0 \end{cases}$

- 若 $y = 1$ 且预测 $f(x) = 1$, 则损失 = 0 (正确且有把握, 损失小) 。
- 若 $y = 1$ 且预测 $f(x) = 0$, 则损失 $\rightarrow \infty$ (错误且置信度高, 惩罚非常大) 。
- 若 $y = 0$ 且预测 $f(x) = 0$, 则损失 $\rightarrow 0$ (正确且有把握, 损失小) 。
- 若 $y = 0$ 且预测 $f(x) = 1$, 则损失 = ∞ (错误且置信度高, 惩罚非常大) 。

大)。



- 这个对数损失 (log loss / cross-entropy 的单样本形式) 在正确预测且置信度高时给小损失；在错误且置信度高时给极大惩罚，从而驱动模型远离“有把握但错误”的预测。
- 两个小图分别表示 $y=1$ 的情况 (Case 1, 损失随 $f(x) \rightarrow 1$ 趋近 0) 和 $y=0$ 的情况 (Case 2, 损失随 $f(x) \rightarrow 0$ 趋近 0)。
- 把两种情况合成一个紧凑统一的损失函数 (因为 y 只有 0 或 1) :
 - 单样本损失 & 总体风险 (平均负对数似然)

$$Loss(f(x), y) = -y \log f(x) - (1 - y) \log(1 - f(x))$$

$$R(\beta) = -\frac{1}{n} \left[\sum_{i=1}^n y \log f(x) + (1 - y) \log(1 - f(x)) \right]$$

- 这是常说的交叉熵 (cross-entropy) 或对数损失 (log loss)。
- 该函数关于参数 β 是凸的 (在常见设定下)，对优化友好，且与最大似然估计等价 (在概率模型视角下)。

Repeat {

Simultaneously update for all β 's

$$\beta_j := \beta_j - \alpha \frac{\partial}{\partial \beta_j} R(\beta)$$

}

After some calculus:

Repeat {

Simultaneously update for all β 's

$$\beta_j := \beta_j - \alpha \sum_{i=1}^n (f(x) - y) x_j$$

}

Note: Same as linear regression BUT with the new function f .

Training data: "examples" x with "labels" y .

$$(x_1, y_1), \dots, (x_n, y_n), x_i \in \mathbb{R}^d$$

回归

- **Regression:** y is a real value, $y \in \mathbb{R}$.

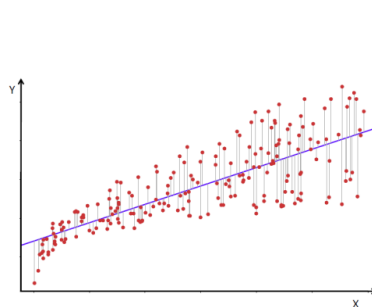
回归器

$$f: \mathbb{R}^d \rightarrow \mathbb{R} \quad (f \text{ is called a regressor})$$

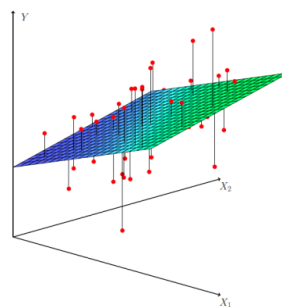
Example: amount of credit, weight of fruit.

- **线性回归 Linear Regression:**

- 定义: 如果回归模型可由线性函数表示, 则称为线性回归。
- $d = 1$: 在 $\mathbb{R}^2$ 中的一条直线 // $d = 2$: 在 $\mathbb{R}^3$ 中的一个超平面



$d = 1$, line in $\mathbb{R}^2$



$d = 2$, hyperplane in $\mathbb{R}^3$

- 线性回归模型:
 - β 称为参数parameters、系数coefficients或权重weights

$$f(x) = \beta_0 + \sum_{j=1}^d \beta_j x_j \quad \text{with } \beta_j \in \mathbb{R}, \quad j \in \{1, \dots, d\}$$

- 估计 (最小二乘法):

- 使用最小二乘损失:

$$\text{loss}(y_i, f(x_i)) = (y_i - f(x_i))^2$$

- 我们希望对所有样本最小化损失, 即最小化风险 (或代价) 函数 R :

$$R = \frac{1}{2n} \sum_{i=1}^n (y_i - f(x_i))^2$$

•

Linear Regression

A simple case with one feature ($d = 1$): $f(x) = \beta_0 + \beta_1 x$

We want to minimize: $R = \frac{1}{2n} \sum_{i=1}^n (y_i - f(x_i))^2$

$$R(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Find β_0 and β_1 that minimize:

$$R(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Find β_0 and β_1 that minimize: $\operatorname{argmin}_{\beta} \left(\frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \right)$

Minimize: $R(\beta_0, \beta_1)$, that is: $\frac{\partial R}{\partial \beta_0} = 0$ $\frac{\partial R}{\partial \beta_1} = 0$

$$\frac{\partial R}{\partial \beta_0} = 2 \times \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \times \frac{\partial}{\partial \beta_0} (y_i - \beta_0 - \beta_1 x_i)$$

$$\frac{\partial R}{\partial \beta_0} = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \times (-1) = 0$$

$$\beta_0 = \frac{1}{n} \sum_{i=1}^n y_i - \beta_1 \frac{1}{n} \sum_{i=1}^n x_i$$

9

Linear Regression

$$\frac{\partial R}{\partial \beta_1} = 2 \times \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \times \frac{\partial}{\partial \beta_1} (y_i - \beta_0 - \beta_1 x_i)$$

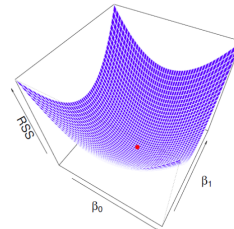
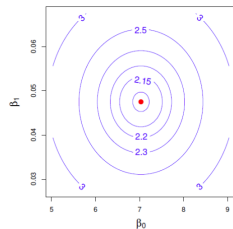
$$\frac{\partial R}{\partial \beta_1} = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \times (-x_i) = 0$$

$$\beta_1 \sum_{i=1}^n x_i^2 = \sum_{i=1}^n x_i y_i - \sum_{i=1}^n \beta_0 x_i$$

Plugging β_0 and β_1 :

$$\beta_1 = \frac{\sum_{i=1}^n y_i x_i - \frac{1}{n} \sum_{i=1}^n y_i \sum_{i=1}^n x_i}{\sum_{i=1}^n x_i^2 - \frac{1}{n} \sum_{i=1}^n x_i \sum_{i=1}^n x_i}$$

- 可视化线性回归的损失函数（通常是残差平方和 RSS）关于两个参数 β_0 和 β_1 的变化



- 右图（3D 曲面）：曲面呈碗状，表明 RSS 随参数变化构成一个凸的二次函数。碗底的最低点（红点）就是使 RSS 最小的最优参数组合（最小二乘解）。
- 左图（等高线/轮廓图）：每一圈等高线表示相同的 RSS 值（标注的数字是对应的损失值）。等高线向内收敛到红点，红点处 RSS 最小。等高线的形状反映参数之间的相关性：
 - 圆形等高线说明 β_0 和 β_1 对 RSS 的影响相对独立、尺度相近；
 - 拉长的椭圆说明参数之间有相关性或尺度差异，沿长轴方向 RSS 变化较小（对解更不敏感），沿短轴方向变化较大（更敏感）。
 - 等高线的密度表示梯度大小：等高线密集的区域梯度大，稀疏处梯度小。

With more than one feature:

$$f(x) = \beta_0 + \sum_{j=1}^d \beta_j x_j$$

Find the β_j that minimize:

$$R = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^d \beta_j x_{ij})^2$$

Let's write it more elegantly with matrices!

• 矩阵表示:

- 令 X 为一个 $n \times (d+1)$ 的矩阵, 每一行以 1 开头, 后面跟着一个特征向量 (用于包含截距 β_0) 。
- 令 y 为训练集的标签向量。
- 令 β 为我们要估计的权重向量。
-

$$X := \begin{pmatrix} 1 & x_{11} & \cdots & x_{1j} & \cdots & x_{1d} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{i1} & \cdots & x_{ij} & \cdots & x_{id} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{n1} & \cdots & x_{nj} & \cdots & x_{nd} \end{pmatrix} \quad y := \begin{pmatrix} y_1 \\ \vdots \\ y_i \\ \vdots \\ y_n \end{pmatrix} \quad \beta := \begin{pmatrix} \beta_0 \\ \vdots \\ \beta_j \\ \vdots \\ \beta_d \end{pmatrix}$$

• 正规方程 / Normal Equation:

- 要找到 $(d+1)$ 个 β 使得 R 最小。

Normal Equation

We want to find $(d+1)$ β 's that minimize R . We write R :

$$R(\beta) = \frac{1}{2n} \|(y - X\beta)\|^2$$

$$R(\beta) = \frac{1}{2n} (y - X\beta)^T (y - X\beta)$$

$$\frac{\partial R}{\partial \beta} = -\frac{1}{n} X^T (y - X\beta)$$

We solve:

$$X^T (y - X\beta) = 0$$

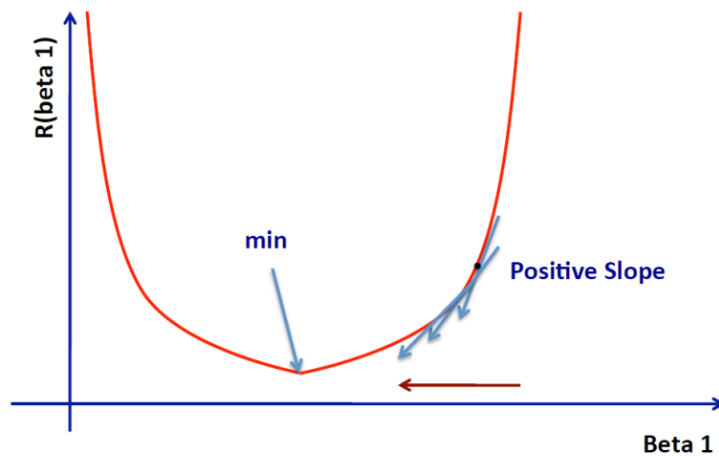
The unique solution is:

$$\beta = (X^T X)^{-1} X^T y$$

$X^T X$ is positive definite.

• 梯度下降 Gradient descent:

- 梯度下降一维示意图: 【横轴是参数 β_1 , 纵轴是代价函数 $R(\beta_1)$ 】



- 红色曲线表示代价随 β_1 的变化，最低点为全局最小值（标为 min）。
- 在曲线上某点处的切线斜率为正（Positive Slope），表示在该点向右移动会增加代价，向左移动会减小代价。
- 梯度的符号告诉你应该增大还是减小参数：斜率为正则减小参数，斜率为负则增大参数。
- 步长由学习率 α 控制；步长太大可能越过最小值，太小则收敛缓慢。

• 梯度下降是一种优化方法

• 重复直到收敛：

- 同时更新所有 β_j (for $j = 0$ 和 $j = 1$) :

$$\beta_0 := \beta_0 - \alpha \frac{\partial}{\partial \beta_0} R(\beta_0, \beta_1)$$

$$\beta_1 := \beta_1 - \alpha \frac{\partial}{\partial \beta_1} R(\beta_0, \beta_1)$$

- α 是学习率（步长）。

• 线性回归中的形式：

- 在线性情形中：

$$\frac{\partial R}{\partial \beta_0} = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \times (-1)$$

$$\frac{\partial R}{\partial \beta_1} = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \times (-x_i)$$

• 重复直到收敛：

- 同时更新所有 β_j (for $j = 0$ 和 $j = 1$) :

•

$$\beta_0 := \beta_0 - \alpha \frac{1}{n} \sum_{i=1}^n (\beta_0 + \beta_1 x_i - y_i)$$

$$\beta_1 := \beta_1 - \alpha \frac{1}{n} \sum_{i=1}^n (\beta_0 + \beta_1 x_i - y_i)(x_i)$$

- 优缺点：

- 解析方法：正规方程（Normal Equation）

- 不需要指定收敛速率或迭代次数。
 - 仅当 $X^T X$ 可逆时适用。
 - 当特征维度 d 很大时，计算 $(X^T X)^{-1}$ 非常慢，复杂度高 $O(d^3)$

- 迭代方法：梯度下降（Gradient Descent）

- 在高维情形下也能有效且高效。
 - 迭代方法（有时需要很多次迭代才能收敛）。
 - 需要选择合适的学习率 α 。

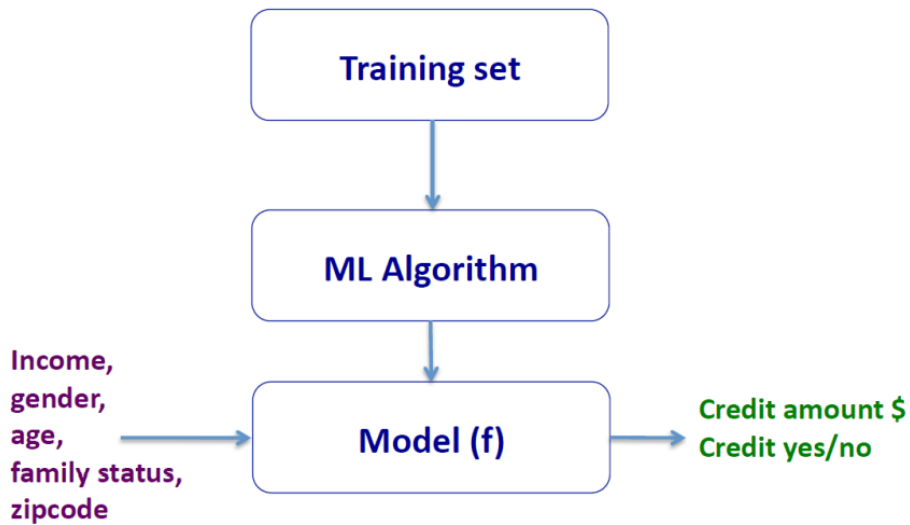
- 实践注意事项：

- 特征缩放：将你的特征缩放到相似的量级，例如

$$x_i := \frac{x_i - \mu_i}{stdev(x_i)}$$

- 说明：缩放可以避免某些特征值过大导致梯度下降收敛慢或不稳定，通常做法是减去均值再除以标准差（标准化）。
 - 学习率：不要使用过小或过大的学习率。
 - 每次迭代后，损失 R 应该下降。
 - 若损失每次下降幅度小于 ϵ ，则宣布收敛。
 - 当 $X^T X$ 不可逆时的原因：
 - a) 相对于样本数量特征太多（例如 50 个样本但 500 个特征）；
 - b) 特征线性相关（例如同时包含以磅为单位和以千克为单位的体重两列）。

- 训练与测试：



- 我们计算 E^{train} ，即样本内误差（训练误差或经验误差/风险）。

$$E^{train}(f) = \sum_{i=1}^n \text{loss}(y_i, f(x_i))$$

- 我们的目标是最小化 $E^{train}(f)$ 。
- 我们希望 $E^{test}(f)$ ，即样本外误差（测试/真实误差），也能很小。
- 损失函数示例：

- 分类误差：

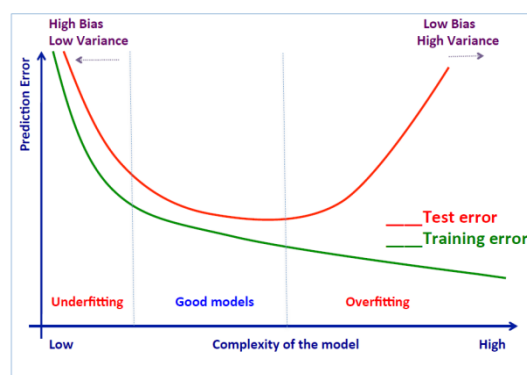
$$\text{loss}(y_i, f(x_i)) = \begin{cases} 1 & \text{if } \text{sign}(y_i) \neq \text{sign}(f(x_i)) \\ 0 & \text{otherwise} \end{cases}$$

- 最小二乘损失：

$$\text{loss}(y_i, f(x_i)) = (y_i - f(x_i))^2$$

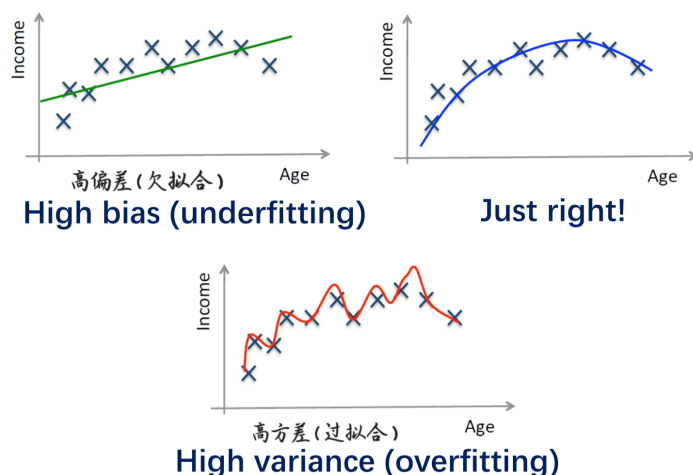
- 结构化风险最小化

Structural Risk Minimization



IMPORTANT

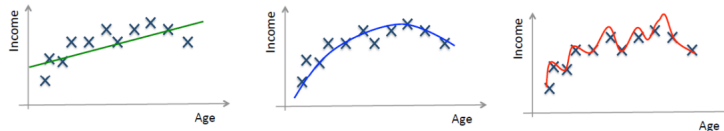
- 这张图展示了随模型复杂度变化的训练误差（绿色）和测试误差（红色）之间的关系，以及偏差-方差权衡。
- 横轴：模型复杂度（从低到高）；纵轴：预测误差（Prediction Error）。
- 绿色曲线（Training error）：表示模型在训练集上的错误率。随着模型复杂度增加，训练误差单调下降（越复杂越能拟合训练数据）。
- 红色曲线（Test error）：表示模型在未见数据上的错误率。它通常先随着复杂度增加而下降（减小偏差），但在复杂度过高时又会上升（方差增大、过拟合），因此呈“U形”或先降后升的趋势。
- 解释偏差与方差：
 - 左侧（低复杂度）：High bias, Low variance（高偏差、低方差）。模型太简单，不能拟合数据中的真实规律，出现欠拟合（underfitting），训练与测试误差都较高。
 - 中间区域：Good models（合适的模型），在该区间有较低的测试误差，达到偏差与方差的平衡。
 - 右侧（高复杂度）：Low bias, High variance（低偏差、高方差）。模型过于复杂，会把训练数据中的噪声也学进去，导致训练误差很低但测试误差上升（过拟合）。
- 重要结论：我们追求在复杂度轴上使测试误差（红线）最小的那个点——既不过于简单也不过于复杂。结构化风险最小化（SRM）和正则化等方法就是为达到这种平衡而设计的。



- 避免过拟合：
 - 手动减少特征数量或进行特征选择。
 - 使用正则化 Regularization（例如，保留特征但通过设置较小的参数值来降低它们的重要性）。
 - 我们要最小化：分类项 + $C \times$ 正则化项

$$\sum_{i=1}^n \text{loss}(y_i, f(x_i)) + C \times R(f)$$

Regularization: Intuition



$$f(x) = \lambda_0 + \lambda_1 x \dots (1)$$

$$f(x) = \lambda_0 + \lambda_1 x + \lambda_2 x^2 \dots (2)$$

$$f(x) = \lambda_0 + \lambda_1 x + \lambda_2 x^2 + \lambda_3 x^3 + \lambda_4 x^4 \dots (3)$$

Hint: Avoid high-degree polynomials.

- 进行交叉验证以估计测试误差。
- 训练、验证与测试 Train, Validation and Test
 - 将数据随机划分为 60% 用于训练，20% 用于验证，20% 用于测试。



- 训练集是一组用于学习模型的样本（例如分类模型）。
- 验证集是一组不能用于训练模型的样本，但可用于调整模型参数（例如在 K-NN 中选择 K）。验证有助于控制过拟合。

• K 折交叉验证 K-fold Cross Validation

- 一种使用训练数据估计测试误差的方法。
- 算法：给定学习算法 A 和数据集 D
 - 步骤 1：将 D 随机划分为 k 个大小相等的子集 $D_1, \dots, D_k$
 - 步骤 2：对于 j = 1 到 k：
 - 在所有 D_i ($i = 1, \dots, k$ 且 $i \neq j$) 上训练算法 A，得到模型 f_j ；
 - 将 f_j 应用于 D_j 并计算在 D_j 上的误差 E^{D_j}
 - 步骤 3：对所有折的误差取平均

$$\sum_{j=1}^k (E^{D_j})$$

- 测试集用于评估最终模型的性能并提供对测试误差的估计。
 - 注意：切勿以任何方式使用测试集来进一步调整参数或修改模型。

		Actual Label	
		Positive	Negative
Predicted Label	Positive	True Positive (TP)	False Positive (FP)
	Negative	False Negative (FN)	True Negative (TN)

Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	The percentage of predictions that are correct
Precision	$TP / (TP + FP)$	The percentage of positive predictions that are correct
Sensitivity (Recall)	$TP / (TP + FN)$	The percentage of positive cases that were predicted as positive
Specificity	$TN / (TN + FP)$	The percentage of negative cases that were predicted as negative

- 准确率 (Accuracy) : 准确率 = (真阳性 + 真阴性) / (真阳性 + 真阴性 + 假阳性 + 假阴性) 正确预测所占的比例
- 精确率 (Precision) : 精确率 = 真阳性 / (真阳性 + 假阳性) 被预测为阳性的预测中实际为阳性的比例
- 灵敏度 / 召回率 (Sensitivity / Recall) : 召回率 = 真阳性 / (真阳性 + 假阴性) 实际为阳性的样本中被预测为阳性的比例
- 特异度 (Specificity) : 特异度 = 真阴性 / (真阴性 + 假阳性) 实际为阴性的样本中被预测为阴性的比例
- 其他评估度量:

- **F₁-Score:**

$$F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

- **F_β-Score:**

$$F_\beta = (1 + \beta^2) \cdot \frac{precision \cdot recall}{(\beta^2 \cdot precision) + recall}$$

- **G-Mean:**

$$G - mean = \sqrt{Precision \times Recall},$$